BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐH SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐÀO TẠO CHẤT LƯỢNG CAO

------o0o------

BÁO CÁO ĐỒ ÁN CUỐI KỲ

ỨNG DỤNG NHẬN DẠNG BIỂN BÁO GIAO THÔNG

HCMUTE

Môn học : **Xử lý ảnh số**

Giáo viên hướng dẫn : **PGS.TS Hoàng Văn Dũng**

Mã nhóm thực hiện : **Nhóm 3 - Lớp 05CLC**

Sinh viên thực hiện :

Nguyễn Minh Đức - 20110461

Nguyễn Văn An - 20110434

Hồ Thái Long - 20161332

Thủ Đức, ngày 08 tháng 05 năm 2023

LỜI CẢM ƠN

Để hoàn thành tốt đề tài và bài báo cáo này, nhóm xin gửi lời cảm ơn chân thành đến giảng viên PGS.TS Hoàng Văn Dũng người đã trực tiếp hỗ trợ nhóm trong suốt quá trình làm đề tài. Nhóm cảm ơn thầy đã đưa ra những lời khuyên và hỗ trợ kịp lúc để định hướng cho nhóm đi đúng với yêu cầu của đề tài đã chọn.

Đề tài và bài báo cáo được nhóm thực hiện trong khoảng thời gian khá ngắn, với những kiến thức còn hạn chế cùng nhiều hạn chế khác về mặt kỹ thuật và kinh nghiệm trong việc thực hiện một ứng dụng nhận dạng. Do đó, trong quá trình làm nên đề tài có những thiếu sót là điều không thể tránh khỏi nên nhóm rất mong nhận được những ý kiến đóng góp quý báu của thầy để nhóm được hoàn thiện hơn về vốn kiến thức và nhóm có thể làm tốt hơn nữa trong những lần sau.

Cuối lời, nhóm kính chúc thầy luôn dồi dào sức khỏe và thành công hơn nữa trong sự nghiệp giảng dạy. Một lần nữa nhóm xin chân thành cảm ơn thầy!

MỤC LỤC

LỜI CẢM ƠN **1**
DANH MỤC HÌNH ẢNH **3**
CHƯƠNG 1. MỞ ĐẦU **5**
1.1 Lý do chọn đề tài 5
1.2 Yêu cầu đặt ra 5
1.3 Khảo sát, đánh giá các hệ thống liên quan 6
1.4 Hướng giải quyết 6
1.5 Xây dựng bộ dữ liệu 7
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT **11**
2.1 Tổng quan về YOLOv5 11
2.2 Tổng quan về CNN 12
2.2.1 Convolutional trong CNN 12
2.2.2 Feature trong CNN 12
2.2.3 Những lớp cơ bản của mạng CNN 13
2.2.3.1 Convolutional layer 13
2.2.3.2 Relu Layer 13
2.2.3.3 Pooling layer 14
2.2.3.4 Fully connected layer 14
2.2.4 Mô hình CNN của nhóm 14
CHƯƠNG 3: XÂY DỰNG ỨNG DỤNG **16**
3.1 Train YOLOV5 16
3.1.1 Môi trường 16
3.1.2 Tiến hành 16
3.2 Train CNN 18
3.2.1 Môi trường 18
3.2.2. Tiến hành 18
3.3 Xây dựng giao diện 20
CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ **21**
4.1 Thực nghiệm 21
4.2 Demo ứng dụng 21
4.2.1 Detect Image 21
4.2.2 Detect Video 22
4.2.3 Detect Live Camera 23
4.3 Phân tích kết quả 24
CHƯƠNG 5: KẾT LUẬN **27**
5.1 Đánh giá kết quả 27
5.1.1 Ưu điểm 27
5.1.2 Nhược điểm 27
5.2 Hướng phát triển 27
TÀI LIỆU THAM KHẢO **28**

DANH MỤC HÌNH ẢNH

Hình	Tên
1	Hướng giải quyết của ứng dụng
2	Thống kê số lượng biển báo
3	Các mô hình và thông số cơ bản của YOLOv5
4	Convolutional layer
5	Pooling layer
6	Hàm create_model
7	Hàm compile_model
8	Cấu trúc data sau khi giải nén
9	Code đổi giá trị số định dạng phân loại trong file thành 0
10	Cây thư mục data chứa dữ liệu hình ảnh và nhãn
11	Code cài đặt Yolov5
12	Đặt duy nhất một class name, với 0 là biển báo
13	Log sau khi train
14	File best.pt nhận được
15	Cấu trúc data sau khi giải nén
16	Code thực hiện cắt ảnh theo đối tượng và phân loại vào các folder
17	Code train CNN
18	Giao diện của ứng dụng
19	Một số hình ảnh nhóm chụp và quay được
20	Detect hình ảnh biển báo
21	Detect video biển báo
22	Nhập địa chỉ IP của camera
23	Detect live camera biển báo

24	Phát hiện, nhận dạng thành công khi có một biển báo
25	Phát hiện, nhận dạng thành công khi có nhiều biển báo
26	Ứng dụng bỏ sót biển
27	Nhận dạng nhầm
28	Biển báo kèm lưu ý

CHƯƠNG 1. MỞ ĐẦU

1.1 Lý do chọn đề tài

Việt Nam là một trong những quốc gia có mật độ giao thông đường bộ cao và đang phát triển nhanh chóng. Với sự tăng trưởng đáng kể trong số lượng phương tiện, hệ thống đường xá đã được nâng cấp và hiện đại hơn để đáp ứng nhu cầu vận chuyển ngày càng tăng. Tuy nhiên, việc vi phạm luật giao thông vẫn là một vấn đề đáng lo ngại. Nhiều người thường chú tâm quá nhiều vào việc điều khiển phương tiện mà quên mất sự hiện diện của biển báo giao thông. Điều này không chỉ gây ra những thiệt hại về kinh tế mà còn có thể gây ra những tai nạn đáng tiếc. Vì vậy, nhóm đề xuất mô hình nhận diện biển báo giao thông trong quá trình tham gia giao thông, đó là một giải pháp hiệu quả để giảm thiểu rủi ro giao thông và giúp người tham gia giao thông tăng cường sự tập trung và chú ý đến các biển báo giao thông. Mô hình này sẽ giúp nhận diện các biển báo giao thông trên đường và cung cấp thông tin về tốc độ giới hạn, hướng đi, đường cấm, và những biển báo khác. Nhờ đó, người tham gia giao thông sẽ có thể điều khiển phương tiện một cách an toàn và đúng quy định hơn, giảm thiểu rủi ro giao thông và đảm bảo an toàn cho mọi người. Trong tương lai, mô hình nhận diện biển báo giao thông có thể được tích hợp vào các phương tiện tự lái hoặc các ứng dụng di động để cung cấp thông tin về giao thông cho người sử dụng. Điều này sẽ giúp tăng cường an toàn và giảm thiểu rủi ro giao thông trên đường, đồng thời cải thiện trải nghiệm của người dùng khi điều khiển phương tiện.

1.2 Yêu cầu đặt ra

- Độ chính xác: Ứng dụng phải đảm bảo độ chính xác cao trong việc nhận dạng các biển báo giao thông để tránh các tình huống nhầm lẫn có thể gây nguy hiểm.
- Tốc độ xử lý: Ứng dụng cần có tốc độ xử lý ổn để phản hồi kịp thời cho người dùng.
- Tính năng phân loại: Ứng dụng cần có khả năng phân loại các biển báo giao thông theo từng loại để giúp người dùng dễ dàng tìm kiếm thông tin cần thiết.
- Giao diện thân thiện: Ứng dụng cần có giao diện thân thiện, dễ sử dụng đối với người dùng không rành công nghệ.

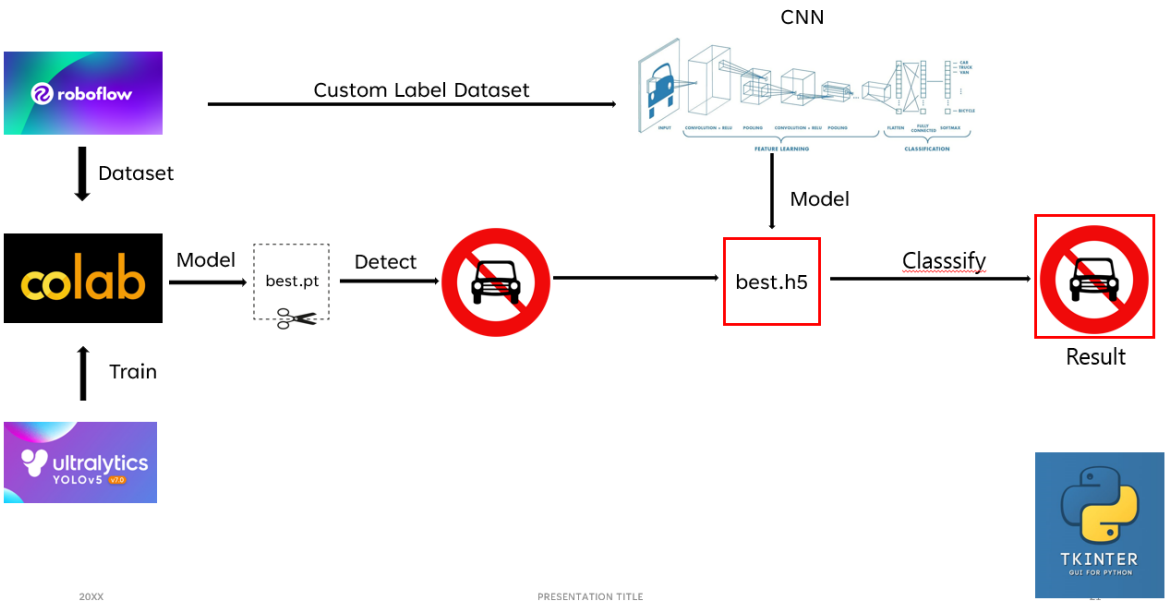
1.3 Khảo sát, đánh giá các hệ thống liên quan

Nhóm có khảo sát Đồ án cuối kỳ môn Máy học - Bài toán nhận diện biển báo giao thông trong khu vực TP.HCM , nhóm LHH, trường ĐH Công Nghệ Thông Tin - ĐHQG TPHCM. [1]
Công nghệ được sử dụng là YOLOv4 và YOLOv5 để nhận dạng và phân loại biển báo giao thông.

Hạn chế:

- Một số loại biển báo không nhận dạng được
- Một số loại biển báo có hình dáng và màu sắc gần giống nhau nên dễ bị nhầm lẫn
- Yếu tố môi trường bên ngoài làm cho model nhận diện không tốt
- Vào ban đêm, ảnh chụp không được rõ nét do ảnh hưởng bởi ánh sáng đèn đường, chất lượng camera chụp ảnh. Do đó có khả năng sai cao hơn so với ban ngày.

1.4 Hướng giải quyết



Hình 1: Hướng giải quyết của ứng dụng

Tổng quan:

- Bước 1: Thu thập data
- Bước 2: Sử dụng YOLOv5 để "Detect" biển báo giao thông
- Bước 3: Crop ảnh biển báo giao thông
- Bước 4: Sử dụng CNN để "Classify" biển báo giao thông

1.5 Xây dựng bộ dữ liệu

Cách thức thu thập: Lấy từ roboflow và các nguồn liên quan

```
./dataset/train\Cam di nguoc chieu  có:  259
./dataset/train\Cam do xe  có:  624
./dataset/train\Cam do xe ngay chan  có:  339
./dataset/train\Cam do xe ngay le  có:  231
./dataset/train\Cam o to con re phai  có:  180
./dataset/train\Cam o to con re trai  có:  198
./dataset/train\Cam quay dau va re  có:  378
./dataset/train\Cam re phai  có:  498
./dataset/train\Cam re trai  có:  481
./dataset/train\Cam xe may va xe 3 banh  có:  742
./dataset/train\Cam xe o to tai  có:  666
./dataset/train\Cau thang bo  có:  262
./dataset/train\Chi dan can phai di  có:  239
./dataset/train\Cho dung do xe buyt  có:  421
./dataset/train\Cho ngoac nguy hiem vong ben phai  có:  266
./dataset/train\Cho ngoac nguy hiem vong ben trai  có:  262
./dataset/train\Di theo vong xuyen  có:  434
./dataset/train\Lan duong danh cho o to tai  có:  783
./dataset/train\Lan duong danh cho o to tai va xe may  có:  499
./dataset/train\Lan duong danh cho xe may  có:  423
./dataset/train\Nguy hiem giao nhau voi duong khong uu tien  có:  473
./dataset/train\Nguy hiem phia truoc co nguoi bang qua duong  có:  479
./dataset/train\Nguy hiem phia truoc co tre em bang qua duong  có:  532
./dataset/train\Noi danh cho nguoi bang qua duong  có:  513
./dataset/train\Toc do toi da 40kmh  có:  105
./dataset/train\Toc do toi da 50kmh  có:  588
./dataset/train\Toc do toi da 60kmh  có:  586
./dataset/train\Toc do toi da 70kmh  có:  210
./dataset/train\Toc do toi da 80kmh  có:  256
Số lượng file ảnh jpg là: 11927 với 29 lớp
```

Hình 2: Thống kê số lượng biển báo

Số lượng file ảnh thu thập được: 11927 ảnh và 29 lớp

STT	Tên biển báo	Hình ảnh	Số lượng (ảnh)
1	Cấm đi ngược chiều		259
2	Cấm đỗ xe		624
3	Cấm đỗ xe ngày chẵn		339
4	Cấm đỗ xe ngày lẻ		231

5	Cấm ôtô con rẽ phải		180
6	Cấm ôtô con rẽ trái		198
7	Cấm quay đầu		378
8	Cấm rẽ phải		498
9	Cấm rẽ trái		481
10	Cấm xe máy và phương tiện thô sơ		742
11	Cấm xe ô tô tải		666
12	Cầu thang dành cho người đi bộ		262
13	Đi vòng chướng ngại		239

14	Điểm dừng xe buýt		421
15	Chỗ ngoặt nguy hiểm bên phải		266
16	Chỗ ngoặt nguy hiểm bên trái		262
17	Đi theo vòng xuyến		434
18	Tốc độ tối đa 40km/h		105
19	Tốc độ tối đa 50km/h		588
20	Tốc độ tối đa 60km/h		586
21	Tốc độ tối đa 70km/h		210
22	Tốc độ tối đa 80km/h		256

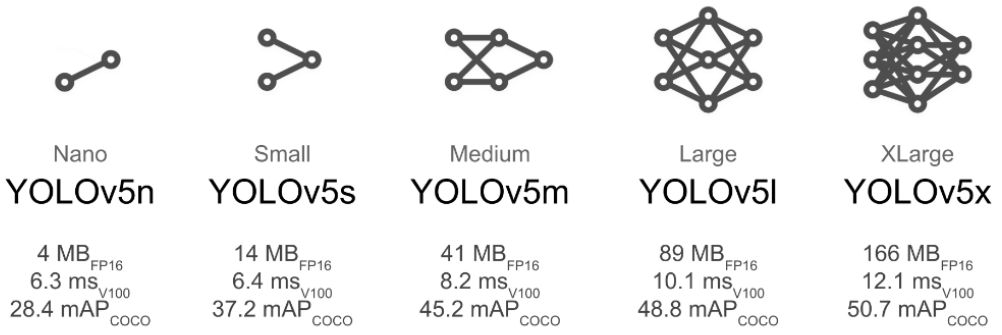
23	Nơi dành cho người sang đường		513
24	Làn đường dành cho oto tải		783
25	Làn đường dành cho oto tải, xe máy		499
26	Làn đường dành cho xe máy		423
27	Giao nhau với đường không ưu tiên		473
28	Phía trước có người băng qua đường		479
29	Phía trước có trẻ em		532

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về YOLOv5

Github repo: [2]

Yolov5 là một mô hình mạng nơ-ron sâu (deep neural network) được sử dụng để phát hiện đối tượng (object detection) trong các hình ảnh và video. Mô hình này được xây dựng dựa trên cấu trúc của mô hình YOLO (You Only Look Once), với nhiều cải tiến và tối ưu hóa để đạt được độ chính xác cao hơn và tốc độ xử lý nhanh hơn. Yolov5 sử dụng kiến trúc mạng nơ-ron sâu làm cơ sở, với các lớp tích chập (convolutional layer) và pooling layer để trích xuất đặc trưng của hình ảnh. Sau đó, các lớp fully connected layer được sử dụng để dự đoán các giá trị bounding box và xác suất của các đối tượng có trong hình ảnh.



Hình 3: Các mô hình và thông số cơ bản của YOLOv5

Yolov5 có nhiều phiên bản khác nhau, mỗi phiên bản có kiến trúc khác nhau và thích hợp cho các ứng dụng khác nhau với hiệu suất tăng dần, bao gồm:

- Yolov5n
- Yolov5s
- Yolov5m
- Yolov5l
- Yolov5x

YOLOv5 cũng khác biệt so với các phiên bản trước của nó do được thực hiện trong Pytorch thay vì một nhánh từ Darknet gốc. Đối với các nhiệm vụ phát hiện đối tượng, mục tiêu là phân loại các đối tượng và trả về các hộp giới hạn của chúng. Thuật toán YOLO chia hình ảnh đầu vào thành một lưới. Đối với mỗi ô, nhiều dự đoán hộp giới hạn của đối tượng được thực hiện liên quan đến ô. Ngoài các tham số của các hộp giới hạn, mỗi hộp còn có xác suất tồn tại một đối tượng trong ô và dự đoán lớp của nó. Đầu ra của các thuật toán YOLO là một tensor với các dự đoán sau:

[pc, bx, by, bh, bw, c1, c2, c3, …, cn]

Pc là xác suất của ô chứa đối tượng, bx, by, là tọa độ x và y của góc trên cùng bên trái của hộp giới hạn và bh, bw là chiều rộng và chiều cao của hộp giới hạn. Phụ thuộc vào số lớp mà mô hình cần dự đoán, có các lớp c1 đến cn tương ứng với n nhãn trong
tensor. Đây là các thông số cho một anchor box. Nếu có một anchor box khác, sẽ có một bộ tham số thứ hai tương tự như cho anchor box thứ hai đó. Số lượng anchor box là một siêu thông số được xác định bởi người dùng. Ưu điểm của phương pháp phát hiện một giai đoạn này là có thời gian phát hiện nhanh hơn nhiều so với các phương pháp hai giai đoạn như Fast R-CNN, Faster R-CNN, Mask R-CNN. Điều này là do
thay vì đưa ra dự đoán trên nhiều vùng của một hình ảnh, YOLO chuyển toàn bộ hình ảnh cùng một lúc vào mạng. Tuy nhiên, nhược điểm của nó là nó có độ chính xác của detection và localization kém hơn so với các phương pháp hai giai đoạn.

2.2 Tổng quan về CNN

Mạng CNN (Convolutional Neural Network) là một loại mạng nơ-ron nhân tạo được thiết kế để xử lý và phân loại hình ảnh.

2.2.1 Convolutional trong CNN

Đây là một "cửa sổ" sử dụng trượt trên ma trận nhằm lấy được những thông tin chính xác và cần thiết nhất. Convolution hay nhân tích chập là cách mà những lớp Convolutional này nhân những phần tử trong ma trận. Sliding Window hay kernel là dạng ma trận có kích thước nhỏ, sử dụng trong nhân tích chập với ma trận hình ảnh.

2.2.2 Feature trong CNN

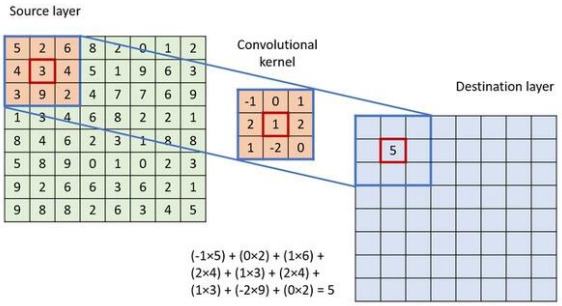
Feature là đặc trưng, mạng CNN sẽ so sánh dựa vào từng mảnh và các mảnh như vậy được gọi là feature. Thay vì phải tiến hành khớp các bức ảnh lại với nhau thì mạng CNN sẽ xác định được sự tương đồng thông qua tìm kiếm thô những đặc trưng khớp với nhau bằng hai hình ảnh tốt hơn.

2.2.3 Những lớp cơ bản của mạng CNN

2.2.3.1 Convolutional layer

Lớp này là phần quan trọng nhất của toàn mạng CNN, nó có nhiệm vụ thực thi các tính toán. Các yếu tố quan trọng trong lớp Convolutional là: padding, stride, feature map và filter map.

- Mạng CNN sử dụng filter để áp dụng vào các vùng của ma trận hình ảnh. Các filter map là các ma trận 3 chiều, bên trong đó là những tham số và chúng được gọi là parameters.
- Stride tức là bạn dịch chuyển filter map theo từng pixel dựa vào các giá trị từ trái qua phải.
- Padding: Thường, giá trị viền xung quanh của ma trận hình ảnh sẽ được gán các giá trị 0 để có thể tiến hành nhân tích chập mà không làm giảm kích thước ma trận ảnh ban đầu.
- Feature map: Biểu diễn kết quả sau mỗi lần feature map quét qua ma trận ảnh đầu vào. Sau mỗi lần quét thì lớp Convolutional sẽ tiến hành tính toán.



Hình 4: Convolutional layer

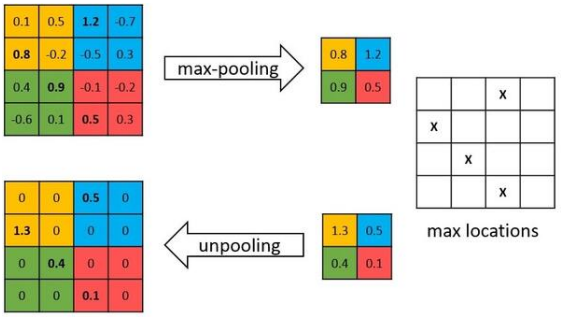
2.2.3.2 Relu Layer

Lớp ReLU này là hàm kích hoạt trong mạng CNN, được gọi là activation function. Nó có tác dụng mô phỏng những nơ ron có tỷ lệ truyền xung qua axon.

Hàm này được sử dụng cho những yêu cầu huấn luyện mạng nơ ron với những ưu điểm nổi bật điển hình là hỗ trợ tính toán nhanh hơn. Trong quá trình dùng hàm ReLU, bạn cần chú ý đến việc tùy chỉnh những learning rate và dead unit.

2.2.3.3 Pooling layer

Khi ma trận ảnh đầu vào có kích thước quá lớn, các lớp Pooling layer sẽ được đặt vào giữa những lớp Convolutional để làm giảm những parameters. Hiện, hai loại lớp Pooling được sử dụng phổ biến là Max pooling và Average.



Hình 5: Pooling layer

2.2.3.4 Fully connected layer

Đây là lớp có nhiệm vụ đưa ra kết quả sau khi hai lớp Convolutional và Pooling đả nhận được ảnh truyền. Khi này, ta sẽ thu được một model đọc được thông tin của ảnh. Để có thể liên kế chúng cũng như cho nhiều đầu ra hơn ta sẽ sử dụng Fully connected layer.

2.2.4 Mô hình CNN của nhóm

```
def create_model():
    vgg16 = tf.keras.applications.VGG16(
        include_top=False, weights='imagenet', input_shape=[HEIGHT, WIDTH, 3])

    x = vgg16.output
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dropout(0.3)(x)
    x = tf.keras.layers.Dense(64)(x)
    x = tf.keras.layers.Dense(128)(x)
    x = tf.keras.layers.LeakyReLU(alpha=0.2)(x)
    x = tf.keras.layers.GaussianDropout(0.4)(x)
    outputs = tf.keras.layers.Dense(
        NUM_CLASSES, activation="softmax", dtype='float32')(x)

    model = tf.keras.Model(vgg16.input, outputs)
    return model
```

Hình 6: Hàm create_model

Hàm create_model(), Sử dụng mô hình mạng nơ-ron cho việc phân loại ảnh sử dụng kiến trúc VGG16 làm cơ sở

Hàm bắt đầu bằng việc khởi tạo mô hình VGG16 từ thư viện Keras, với tham số include_top được đặt là False, có nghĩa là lớp cuối cùng của mô hình VGG16 đã được huấn luyện trước, chịu trách nhiệm cho việc phân loại, sẽ không được bao gồm trong mô hình mới. Tham số weights được đặt là

'imagenet', có nghĩa là trọng số đã được huấn luyện trên tập dữ liệu ImageNet sẽ được sử dụng.
Tiếp theo, đầu ra của mô hình VGG16 được truyền qua một lớp pooling toàn cục trung bình, trung bình hóa mỗi bản đồ đặc trưng trong tensor đầu ra, cho kết quả một đầu ra có kích thước cố định. Sau đó là một lớp dropout, loại bỏ ngẫu nhiên 30% các kích hoạt để ngăn ngừa quá khớp.
Đầu ra của lớp dropout sau đó được truyền qua hai lớp kết nối đầy đủ với 64 và 128 đơn vị tương ứng. Một hàm kích hoạt LeakyReLU với alpha là 0,2 được áp dụng cho đầu ra của hai lớp dense để giới thiệu độ không tuyến tính. Một lớp dropout Gaussian với tỷ lệ là 0.4 được áp dụng cho đầu ra của hàm kích hoạt LeakyReLU.
Cuối cùng, đầu ra của lớp dropout Gaussian được truyền qua một lớp dense với NUM_CLASSES đơn vị và hàm kích hoạt softmax, tạo ra một phân phối xác suất trên các lớp có thể có.

```
77  def compile_model(model, lr=0.0001):
78
79      optimizer = tf.keras.optimizers.Adam(lr=1e-4)
80
81      loss = tf.keras.losses.CategoricalCrossentropy()
82
83      metrics = [
84          tf.keras.metrics.CategoricalAccuracy(name='categorical_accuracy')
85      ]
86
87      model.compile(optimizer=optimizer, loss=loss, metrics=metrics)
88
89      return model
```

Hình 7: Hàm compile_model

Hàm compile_model(model, lr = 0.0001), để biên dịch mô hình mạng nơ-ron được tạo ra bởi hàm create_model. Hàm này cũng cho phép cấu hình siêu tham số của mô hình như tốc độ học (lr).
Hàm bắt đầu bằng việc khởi tạo một bộ tối ưu hóa Adam với tốc độ học (lr) được truyền vào như một tham số. Hàm mất (loss) được đặt là phân loại chéo (CategoricalCrossentropy), và chỉ số đánh giá được chọn là độ chính xác phân loại (CategoricalAccuracy).
Sau đó, hàm compile của mô hình Keras được gọi để thiết lập mô hình với các tham số tối ưu hóa, hàm mất và chỉ số đánh giá. Cuối cùng, hàm trả về mô hình đã được biên dịch.

CHƯƠNG 3: XÂY DỰNG ỨNG DỤNG

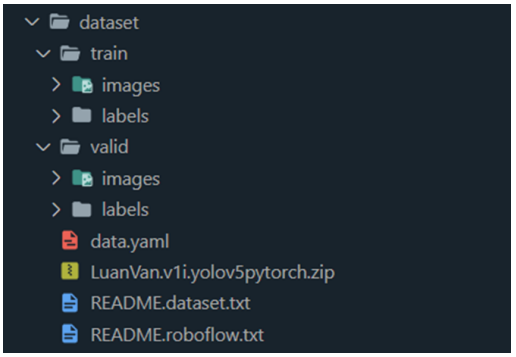
3.1 Train YOLOV5

3.1.1 Môi trường

Sử dụng Google Colab để train. Google Colab cung cấp một số tài nguyên tính toán miễn phí cho người dùng, bao gồm CPU, GPU và TPU, giúp cho việc huấn luyện mô hình trở nên nhanh hơn và hiệu quả hơn. Ngoài ra, người dùng cũng có thể chia sẻ các notebook của mình với người khác và làm việc chung trên cùng một notebook.

3.1.2 Tiến hành

Bước 1: Lấy data từ [3] và giải nén.



Hình 8: Cấu trúc data sau khi giải nén

Gồm tập train và valid, trong đó mỗi folder gồm images và labels. Images chứa các file ảnh có ảnh biển báo, tương ứng với các file ảnh đó là các file txt chứa thông tin lớp phân loại và tọa độ 4 điểm của biển báo và được lưu trữ trong labels.

Bước 2: Mục tiêu train Yolov5 của nhóm chỉ nhận dạng biển báo mà không phân loại nên format các class name thành số 0 với lệnh sau

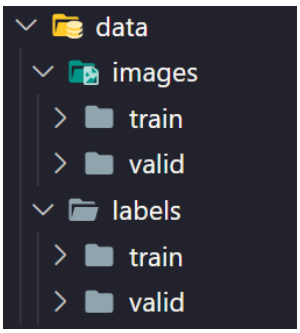
```
def format(path):
    for file_name in os.listdir(path):
        print("+===== ", file_name)
        if file_name.endswith(".txt"):
            file_path = os.path.join(path, file_name)
            with open(file_path, "r") as file:
                lines = file.readlines()

            # Đổi giá trị số định dạng phân loại thành 0
            for i in range(len(lines)):
                if len(lines[i]) > 0:
                    lines[i] = "0 " + " ".join(lines[i].split(" ")[1:])

            with open(file_path, "w") as file:
                file.writelines(lines)
            print(
                f"Đã đổi giá trị số định dạng phân loại trong file {file_name} thành 0.
                ")
    print("END")
```

Hình 9: Code đổi giá trị số định dạng phân loại trong file thành 0

Bước 3: Upload data lên colab theo cấu trúc



Hình 10: Cây thư mục data chứa dữ liệu hình ảnh và nhãn

Bước 3: Chạy đoạn mã sau để cài đặt YOLOv5

```
%%bash
git clone https://github.com/ultralytics/yolov5
cd yolov5
pip install -r requirements.txt
```

Hình 11: Code cài đặt Yolov5

Bước 4: Setup lại file yolov5/data/data.yaml.

data.yaml

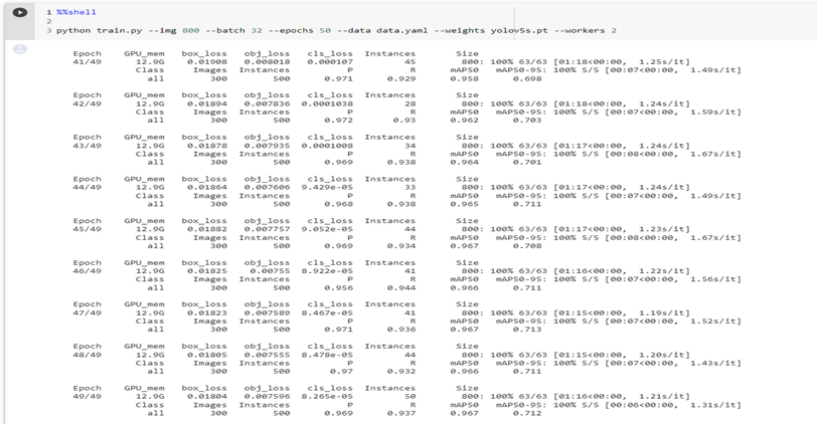
```
train: ../train/images
val: ../valid/images

nc: 1
names: ['0']
```

Hình 12: Đặt duy nhất một class name, với 0 là biển báo

Bước 5: Chạy đoạn mã sau để train với batch size = 32, epochs = 50, với pretrain model yolovs.pt

python train.py --img 800 --batch 32 --epochs 50 --data data.yaml --weights yolov5s.pt --workers 2



Hình 13: Log sau khi train

➢ Kết quả: Nhận được file best.pt



Hình 14: File best.pt nhận được

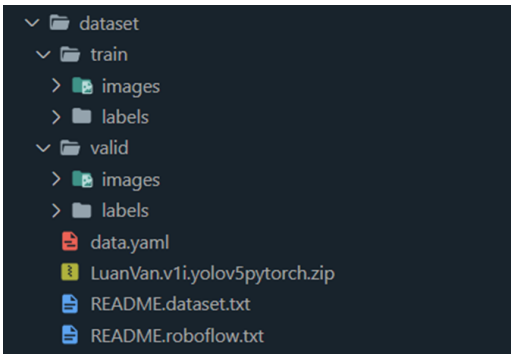
3.2 Train CNN

3.2.1 Môi trường

Sử dụng GPU của máy tính cá nhân để train

3.2.2. Tiến hành

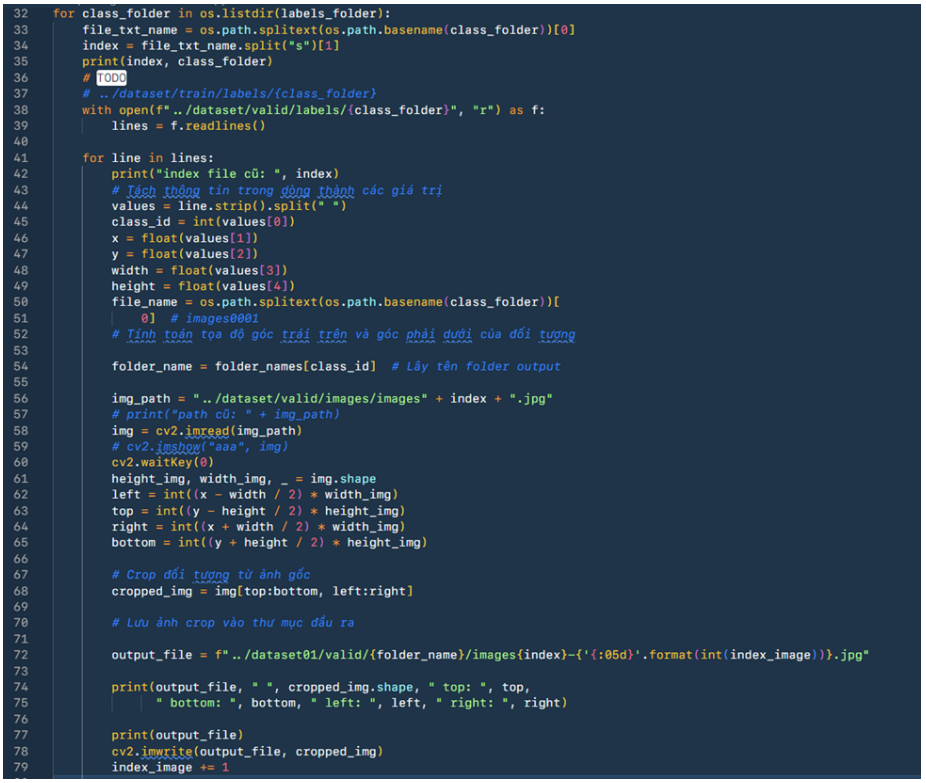
Bước 1: Lấy data từ [3] và giải nén.



Hình 15: Cấu trúc data sau khi giải nén

Gồm tập train và valid, trong đó mỗi folder gồm images và labels. Images chứa các file ảnh có ảnh biển báo, tương ứng với các file ảnh đó là các file txt chứa thông tin lớp phân loại và tọa độ 4 điểm của biển báo và được lưu trữ trong labels.

Bước 2: Cắt ảnh theo đối tượng và phân loại vào các folder theo từng loại để phục vụ train model CNN:



```
for class_folder in os.listdir(labels_folder):
    file_txt_name = os.path.splitext(os.path.basename(class_folder))[0]
    index = file_txt_name.split("s")[1]
    print(index, class_folder)
    # TODO
    # ../dataset/train/labels/{class_folder}
    with open(f"../dataset/valid/labels/{class_folder}", "r") as f:
        lines = f.readlines()

    for line in lines:
        print("index file cũ: ", index)
        # Tách thông tin trong dòng thành các giá trị
        values = line.strip().split(" ")
        class_id = int(values[0])
        x = float(values[1])
        y = float(values[2])
        width = float(values[3])
        height = float(values[4])
        file_name = os.path.splitext(os.path.basename(class_folder))[
            0]  # images0001
        # Tính toán tọa độ góc trái trên và góc phải dưới của đối tượng

        folder_name = folder_names[class_id]  # Lấy tên folder output

        img_path = "../dataset/valid/images/images" + index + ".jpg"
        # print("path cũ: " + img_path)
        img = cv2.imread(img_path)
        # cv2.imshow("aaa", img)
        cv2.waitKey(0)
        height_img, width_img, _ = img.shape
        left = int((x - width / 2) * width_img)
        top = int((y - height / 2) * height_img)
        right = int((x + width / 2) * width_img)
        bottom = int((y + height / 2) * height_img)

        # Crop đối tượng từ ảnh gốc
        cropped_img = img[top:bottom, left:right]

        # Lưu ảnh crop vào thư mục đầu ra

        output_file = f"../dataset01/valid/{folder_name}/images{index}-{'{:05d}'.format(int(index_image))}.jpg"

        print(output_file, " ", cropped_img.shape, " top: ", top,
              " bottom: ", bottom, " left: ", left, " right: ", right)

        print(output_file)
        cv2.imwrite(output_file, cropped_img)
        index_image += 1
```

Hình 16: Code thực hiện cắt ảnh theo đối tượng và phân loại vào các folder

Bước 3: Tiến hành train CNN với kiến trúc như hình: (epoch = 30, batch size = 8)

```
def create_model():
    vgg16 = tf.keras.applications.VGG16(
        include_top=False, weights='imagenet', input_shape=[HEIGHT, WIDTH, 3])

    x = vgg16.output
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dropout(0.3)(x)
    x = tf.keras.layers.Dense(64)(x)
    x = tf.keras.layers.Dense(128)(x)
    x = tf.keras.layers.LeakyReLU(alpha=0.2)(x)
    x = tf.keras.layers.GaussianDropout(0.4)(x)
    outputs = tf.keras.layers.Dense(
        NUM_CLASSES, activation="softmax", dtype='float32')(x)

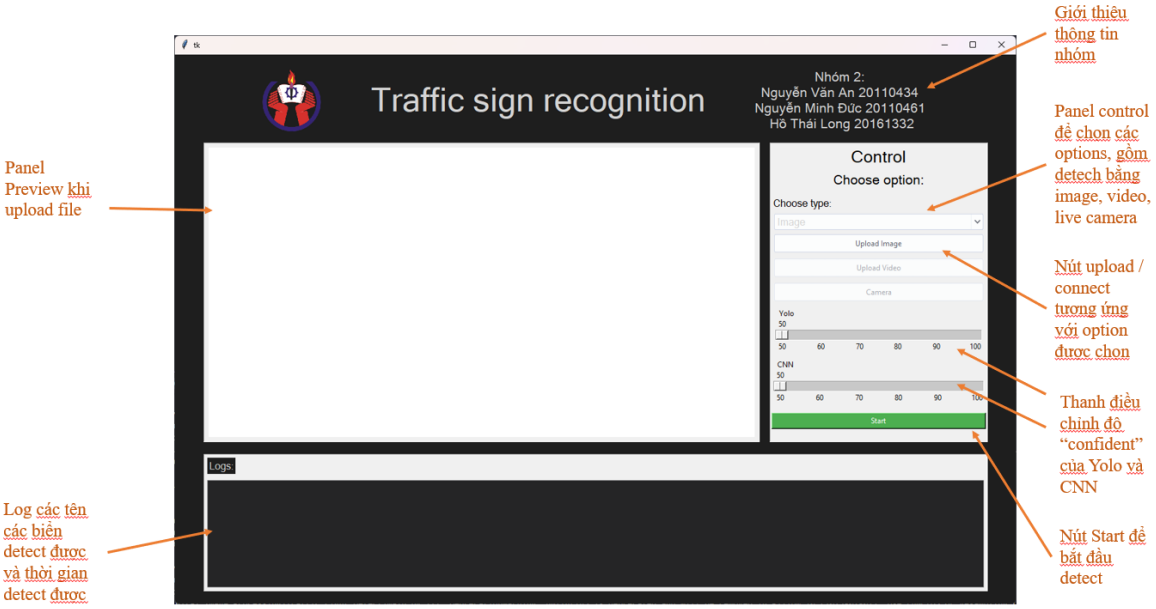
    model = tf.keras.Model(vgg16.input, outputs)
    return model
```

Hình 17: Code train CNN

➢ Kết quả thu được file best.h5

3.3 Xây dựng giao diện

Ứng dụng yêu cầu cần có giao diện đơn giản, thân thiện với người sử dụng nên nhóm quyết định sử dụng **Tkinter** - thư viện GUI tiêu chuẩn cho Python



Hình 18: Giao diện của ứng dụng

CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Thực nghiệm

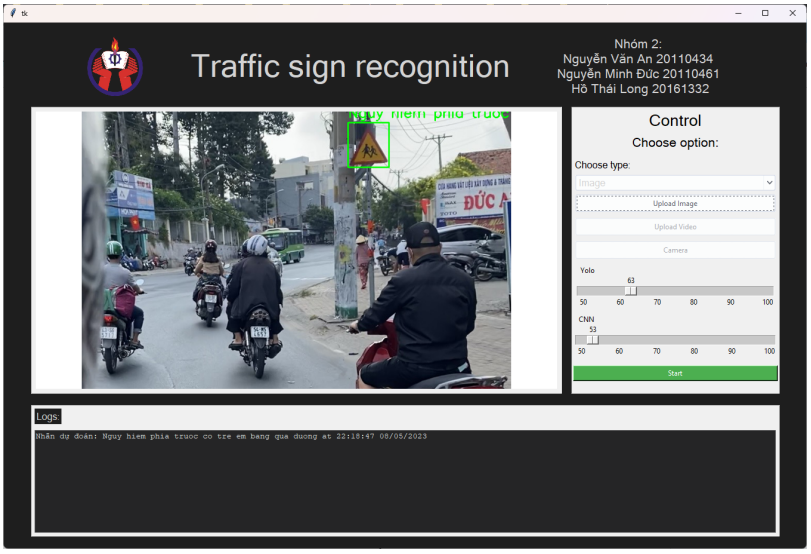
Ngày 6/5/2023 nhóm đi chụp và quay một số hình ảnh biển báo giao thông trên các đường Lê Văn Việt, Kha Vạn Cân, Xa lộ Hà Hội,... để kiểm thử ứng dụng



Hình 19: Một số hình ảnh nhóm chụp và quay được

4.2 Demo ứng dụng

4.2.1 Detect Image



Hình 20: Detect hình ảnh biển báo

Bước 1: Chọn type Image

Bước 2: Nhấn nút Upload Image

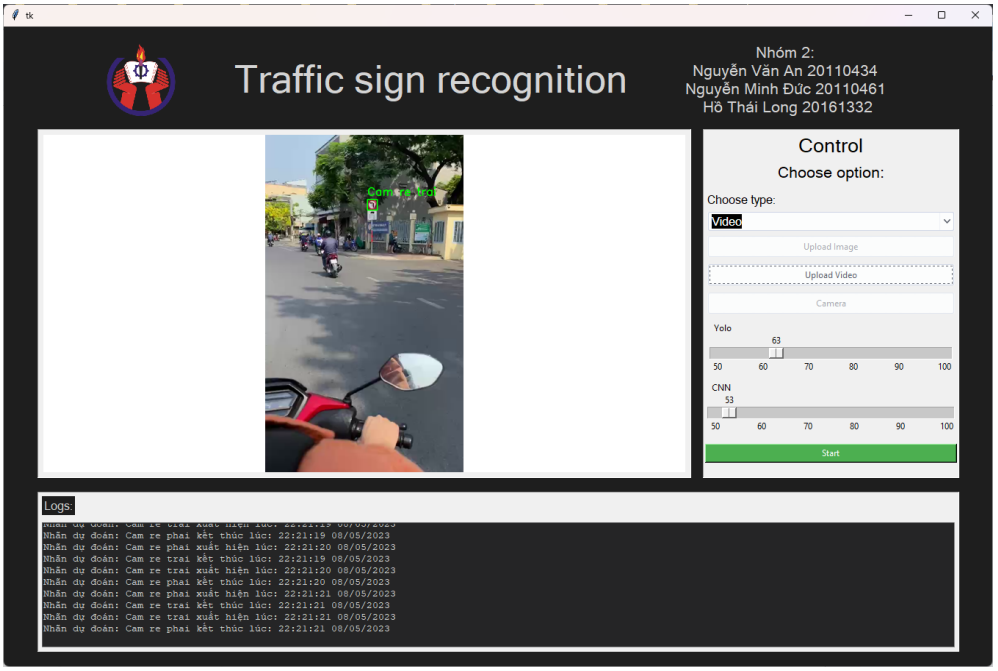
Bước 3: Chọn Image từ máy tính để upload

Bước 4: Điều chỉnh độ “confident” của Yolo và CNN phù hợp

Bước 5: Nhấn nút Start

➢ Kết quả: Biển báo giao thông có trong ảnh sẽ được đóng khung và hiển thị tên loại biển báo, đồng thời log ra tên loại biển báo và thời gian detect được

4.2.2 Detect Video



Hình 21: Detect video biển báo

Bước 1: Chọn type Video

Bước 2: Nhấn nút Upload Video

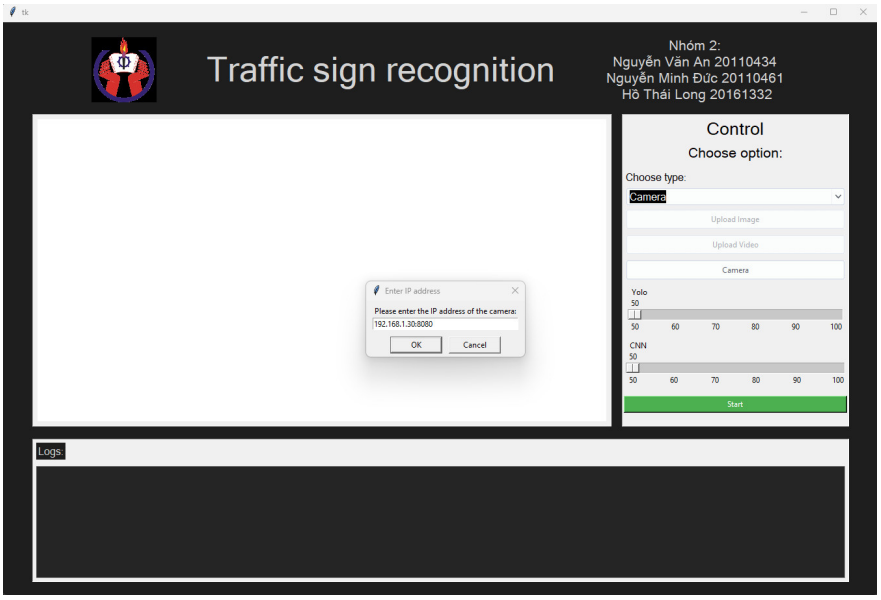
Bước 3: Chọn video từ máy tính để upload

Bước 4: Điều chỉnh độ "confident" của Yolo và CNN phù hợp

Bước 5: Nhấn nút start

➢ Kết quả: Video sẽ được phát, các biển báo có trong video sẽ lần lượt được đóng khung và hiển thị tên loại biển báo, đồng thời lần lượt log ra tên loại biển báo và thời gian detect được

4.2.3 Detect Live Camera



Hình 22: Nhập địa chỉ IP của camera



Hình 23: Detect live camera biển báo

Bước 1: Chọn type Camera

Bước 2: Chọn nút Camera

Bước 3: Điều chỉnh độ "confident" của Yolo và CNN phù hợp

Bước 4: Chọn nút Start

Bước 5: Nhập địa chỉ IP của camera

Bước 6: Chọn nút OK

➢ Kết quả: Camera sẽ được kết nối, các biển báo có mà camera quay được sẽ lần lượt được đóng khung và hiển thị tên loại biển báo, đồng thời lần lượt log ra tên loại biển báo và thời gian detect được

4.3 Phân tích kết quả

- Trong điều kiện lý tưởng, ảnh đầy đủ ánh sáng, ảnh không bị nhòe thì ứng dụng dễ dàng nhận dạng chính xác loại biển báo

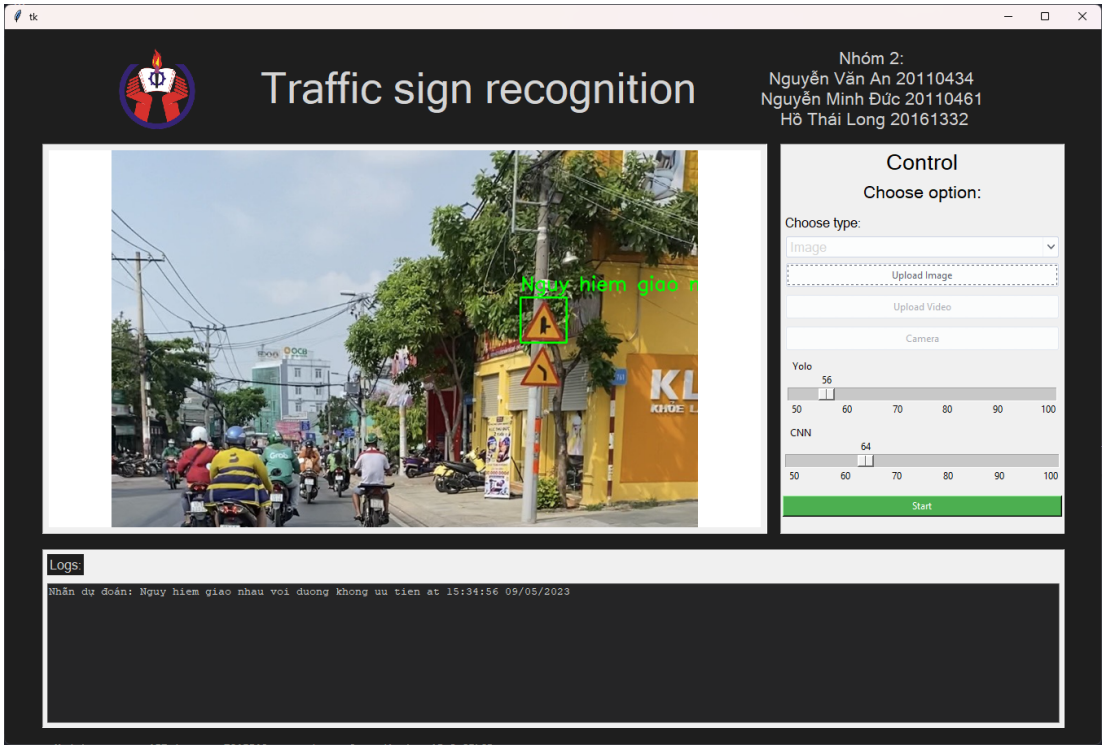


Hình 24: Phát hiện, nhận dạng thành công khi có một biển báo



Hình 25: Phát hiện, nhận dạng thành công khi có nhiều biển báo

- Trong trường hợp có nhiều vật cản ở xung quanh, biển báo có thể bị bỏ sót. Hình bên dưới có 2 biển báo nhưng chỉ detect được 1.



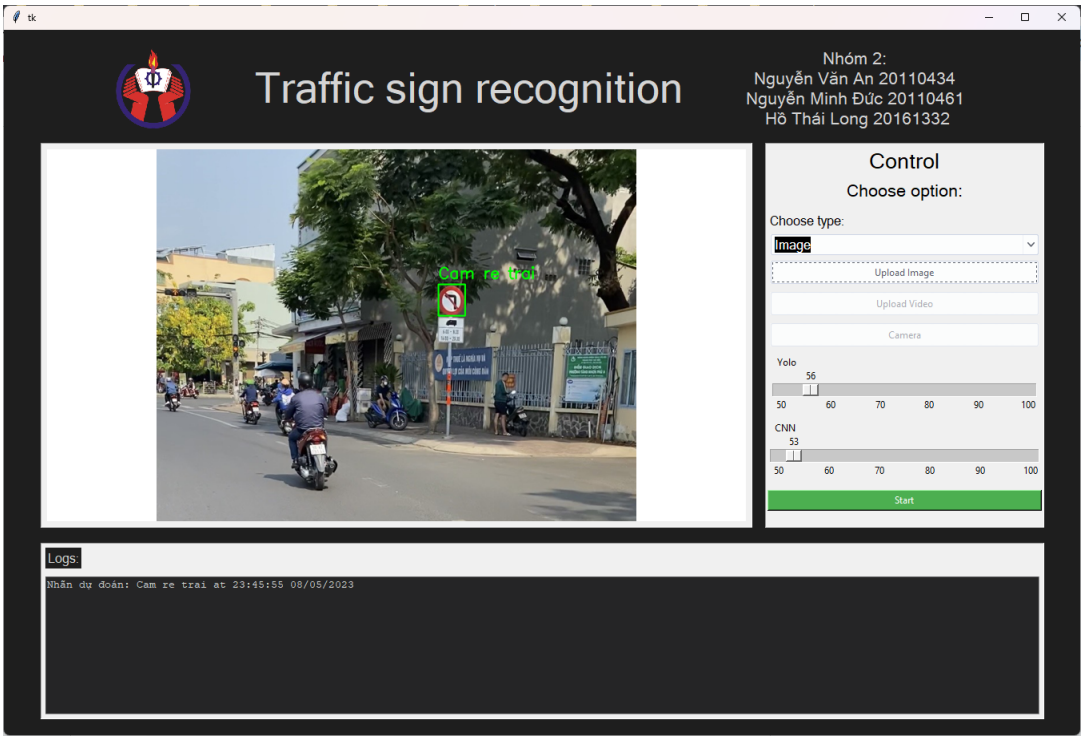
Hình 26: Ứng dụng bỏ sót biển

- Trong trường hợp các biển gần giống nhau, nếu biển không có trong bộ data của nhóm thì sẽ bị nhầm thành biển gần giống nó trong data của nhóm. Hình bên dưới biển "Cấm đi thẳng" bị nhầm thành "Cấm rẽ phải"



Hình 27: Nhận dạng nhầm

- Chưa có khả năng đọc được các lưu ý đi kèm biển báo



Hình 28: Biển báo kèm lưu ý

CHƯƠNG 5: KẾT LUẬN

5.1 Đánh giá kết quả

Về căn bản ứng dụng của nhóm đã đáp ứng đủ yêu cầu đặt ra của một ứng dụng nhận dạng biển báo giao thông. Cùng với đó, nhóm đã hoàn thành đồ án đúng deadline.

5.1.1 Ưu điểm

- Ứng dụng có thể nhận diện được các loại biển báo thông dụng, đáp ứng được hầu hết các tình huống thực tế.
- Trong điều kiện lý tưởng, ứng dụng dễ dàng nhận dạng chính xác các biển báo.
- Ứng dụng hoạt động đa dạng, có thể nhận dạng qua ảnh, video hoặc live camera.
- Giao diện người dùng khá thân diện, người dùng dễ dàng sử dụng ứng dụng mà không cần phải là chuyên gia công nghệ, không cần tìm hiểu nhiều.

5.1.2 Nhược điểm

- Một vài biển báo ít gặp không có trong bộ data, nên thường xảy ra trường hợp vì thế không nhận dạng được hoặc nhận dạng sai.
- Một số trường hợp bị ngoại cảnh tác động dẫn đến nhận dạng sai hoặc bỏ sót.
- Tính năng nhận dạng qua video thường xảy ra tình trạng giật lag khi sử dụng.
- Tính năng nhận dạng qua live camera còn hạn chế là camera và máy tính chạy ứng dụng phải cùng chung một mạng, tốc độ xử lý chưa cao.

5.2 Hướng phát triển

- Thu thập và cập nhật đầy đủ biển báo vô bộ data để ứng dụng hoàn thiện
- Nâng cao độ chính xác nhận diện bằng cách áp dụng thêm các kỹ thuật xử lý ảnh và học máy tiên tiến hơn.
- Thêm chức năng đọc được các lưu ý đi kèm của biển báo
- Tích hợp ứng dụng vô app điện thoại để có thể sử dụng mọi lúc mọi nơi

TÀI LIỆU THAM KHẢO

[1]https://github.com/lynguyenminh/CS114.M11.Vietnam-traffic-sign-detection/tree/master/Final_Project?fbclid=IwAR1jDQKwSEdOokDyYDVanwynbPdIpMKUwlU5j6UfS5tBSgViRdJeR1FXmkg#yolonhanh

[2]https://github.com/ultralytics/yolov5

[3]https://universe.roboflow.com/ds/BAWEVJKraU?key=TDNQ1xtbBV (bộ dataset)

[4]https://aptech.fpt.edu.vn/cnn-la-gi.html